



The Open University

M263 Unit 2

UNDERGRADUATE ICT AND COMPUTING

Building Blocks of Software



Introduction to code

Unit **2**



The Open University

M263 Building Blocks of Software

Unit 2

Introduction to code

Study guide

Sections 2 and 5 of this unit should each take less time than the other sections. There is no practical computing work that is essential to the study of this unit. You can, however, study the first Practical Computing session at any time after you have completed Section 2. This will provide an introduction to the Workpad, which is a computer package that you will use in later units to try out code.

You may already be familiar with some of the ideas introduced in this unit. If so, regard it as an introduction to the rules of the course code. Make sure that after reading this unit you have understood the distinction between specification and implementation, and the idea of a correct implementation, as discussed in Section 4.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2297 0

Contents

Introduction	4
1 Variables and assignment	5
1.1 Assignment	5
1.2 Literals	6
1.3 Variables and type	6
1.4 Comments	9
1.5 Calling functions in code	10
1.6 Invalid code	11
1.7 Reserved words	11
2 Conditionals	12
3 Specification	16
3.1 Specifying a function	17
3.2 Specifying a user-defined process	21
3.3 Checking that function calls are valid	22
4 Implementation	25
4.1 Implementing a user-defined function	25
4.2 Calling a user-defined function in code	30
4.3 Testing correctness	31
5 Repetition in code	35
Summary of Unit 2	42
Solutions to the exercises	43
Index	48

Introduction

In this unit we shall introduce statements expressed in a way suitable for interpretation by a computer. These statements are part of a language and notation that we shall refer to as the **course code**. This code is peculiar to this course. It includes forms of statement that are typical of programming languages of a particular style. These include: **assignment** of a value to a variable, **conditionals**, which are statements whose outcome depends on whether some condition is **true** or **false**, and **loops**, which repeat a group of other statements. The fundamental statements of the course code are introduced in this unit, but some further features of the code are described in later units. In Unit 3, we will consider the types of data available in code. Object-oriented programming languages extend the forms of data and statement beyond those available in older imperative languages. We shall look at these in Unit 4.

Programming languages that use such statements are known broadly as **imperative** languages.

In Unit 1 we introduced processes on data, including functions such as *ASC*, which gives the ASCII code of a character, and binary operations such as addition of integers. When writing code, we will need to call on such processes. To describe a process available in code, we use a **specification**. For a function, the specification conveys the same information as do the function descriptions used in Unit 1. Knowing the specification of a process enables us to determine the effect of code that calls on it. We do not need to know how the computer achieves the effect described in the specification. Indeed, we shall make no attempt to describe how primitive processes such as addition of integers are executed on a computer.

Suppose that you want to write code to fulfil some particular purpose. It is sensible first to provide an overview of the code through a specification. This gives an outside view of the effect of the code, without any need to refer to the details of how this effect is achieved. Code to achieve this effect is called an **implementation** of the specification. Keeping separate the specification (a statement of behaviour) and the implementation (code to achieve that behaviour) is known as **abstraction**. This approach helps you to be clear as to what you want code to do, and to recognise errors.

We shall not deal with large bodies of code in this course, but the techniques described here are important in that context. Abstraction assists when subdividing large bodies of code into smaller modules — code packages of manageable size. We can separate the specification and implementation of each module, and write the code in stages: building modules that call only on primitive processes that the computer can already perform, then building higher-order modules that call on these, and so on.

We use the course code as a vehicle to introduce programming principles. We are not teaching the use of a particular programming language. To maintain attention to principles, the course code makes available fewer statements and forms of data than would be provided in a typical programming language in practice. Also, we shall not emphasise some complicating features that are needed by programming languages to enable them actually to work. For example, in practice, a programming language will need to set a bound on the size of integer that can be stored, but we shall not discuss such points in detail. However, the course code does have clearly defined rules. These rules make it possible to use a computer to follow the effect of statements expressed in the code, as you will see in Unit 3.

In Section 1 we will introduce assignment. Section 2 deals with conditional statements. Section 3 considers specification of a process that returns a value. Section 4 looks at implementations of such processes. Loops are introduced in Section 5.

1 Variables and assignment

This section introduces assignment, which is a basic construct in imperative programming languages.

1.1 Assignment

Suppose that you add up the list of numbers [89,13,42] on a simple hand calculator. You turn the calculator on, then enter the number 89. You then add 13, and then add 42. Each step changes the value shown by the calculator display, which in turn is: 89, then 102, then 144.

Regard the display of the calculator as showing the current value of a variable stored electronically, and call this variable *current*. A sequence of processes has been applied to this variable, which we can write as follows.

```
current ← 89
current ← current + 13
current ← current + 42
```

For the hand calculator, a line such as $current \leftarrow current + 13$ is a description of the process carried out by the calculator when certain keys are pressed. We want now to regard this as a **statement**, by which we mean a piece of code that might be addressed to a computer, to describe a calculation that we want performed.

The assignment statement $current \leftarrow current + 13$ contains several important elements. The first is the idea of a **variable**, which was introduced in Unit 1. We will regard a variable as an area of memory within a computer that has an **identifier** (for example *current*), and where the data stored in the area of memory can change.

So *current* identifies an area of memory where data can be stored. In code, we can refer to the value stored there and we can perform some calculation involving this value, such as adding 13 to it.

The symbol $\leftarrow$ is called the **assignment** symbol. The effect of a statement such as

```
current ← 89
```

is straightforward. The value 89 is stored in *current*. Any value previously stored in *current* is overwritten, and so is no longer available.

The effect of the assignment statement

```
current ← current + 13
```

is a little more complicated, and is as follows. First, 13 is added to the number currently stored in *current*. Then this calculated value is stored in the variable *current*. The old value stored in *current* is written over, and is no longer available.

Different programming languages use different symbols for assignment. Some use $\leftarrow$, some use $=$, and others use $:=$.

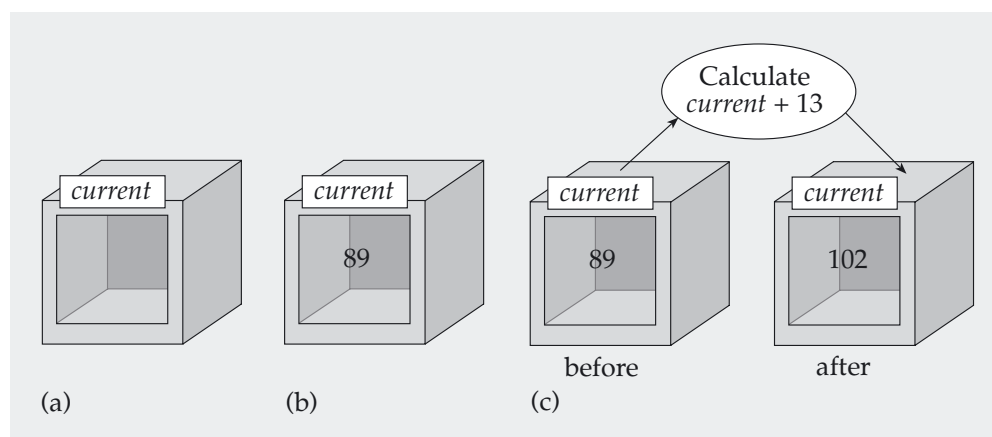


Figure 1.1 (a) A storage location with identifier *current* attached to it. (b) The storage location *current* with the integer 89 stored there. (c) The effect of the assignment $current \leftarrow current + 13$.

You might read the assignment statement $current \leftarrow current + 13$ as “replace *current* by $current + 13$ ”. If you do that, you need to remember that $current + 13$ is calculated using the value stored in *current* before the assignment is executed. So, for example, if *current* stores the value 100 before the assignment $current \leftarrow current + 13$, then *current* will store the value 113 after the statement.

In general, an assignment of the form $current \leftarrow something$ first calculates the expression *something* that appears to the right of $\leftarrow$, using the value stored in *current* before the statement is executed. It then stores the calculated value of *something* in the storage location *current*, and in doing so, throws away whatever was stored in *current* before.

1.2 Literals

To be able to interpret the assignment $current \leftarrow 89$, a computer will need to be able to interpret 89. In writing code, we will assume that a computer is able to interpret such notation for writing an integer value. We refer to a constant value, such as 89, written in a notation that the computer can interpret, as a **literal**. We will also assume that the computer can interpret notation for literal values for other primitive forms of data. We introduced these notations in Unit 1. We write a character literal in single inverted commas, as in ‘%’, for example. We will also assume that a computer can interpret the Boolean literals **true** and **false**.

1.3 Variables and type

If you want to write statements involving a particular variable, such as *current*, the computer needs to set aside an identified storage location where it will store the value which is held by that variable. To allocate a suitable amount of storage, we need to state what form of data will be stored in the variable.

For example, to set aside a region of memory suitable for storing an integer, and attach the identifying label *anInt* to that storage region, we write

```
var anInt in Int.
```

We will say that this statement creates a variable *anInt*, and we refer to *Int* as the **type** of the variable. We must only store integer values in the variable *anInt*. Statements such as $anInt \leftarrow -33$, or $anInt \leftarrow anInt + 3$ will be fine, but writing $anInt \leftarrow 'A'$ would be incorrect, since it attempts to store a character in a variable that is reserved for an integer. This is a rule of the course code, which we will expand on later.

Similarly, to set aside a region of memory *aChar* to store a character, we will write

```
var aChar in Char.
```

To set aside memory for a variable *aBool* to store a Boolean value we write

```
var aBool in Bool.
```

Using a **var** statement to give just the form of data that is being stored is an interim approach. The computer also needs to know what processes may be applied to the variable, but to state that, we need the idea of **data type**, which is discussed in Unit 3.

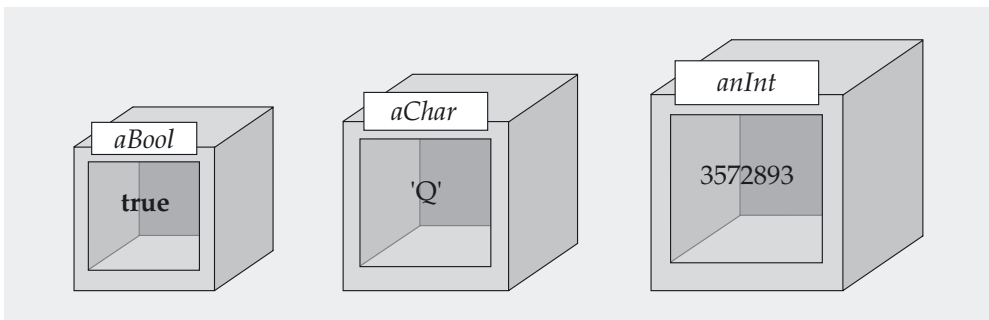


Figure 1.2 The amount of storage required will be different for different forms of data. A Boolean value requires little memory for storage; a character requires more memory, and an integer still more.

In thinking about the amount of memory required to store some value, you may realise that a computer will not be able to store integers of arbitrary size. In practice there will be a limit on the size of integer that can be stored. This limit will be very large, say 2^{32} . So in a programming language, an integer variable like *anInt* will only be able to store values satisfying a condition such as

$$-2^{32} < anInt < 2^{32}.$$

(We will not set a specific limit on the size of an integer permitted in the course code. None of our examples will involve large integers.)

In many programming languages a statement equivalent to **var anInt in Int** would create the variable *anInt* with a particular stored value (usually referred to as the **default** value). For now, though, we will assume that variables are created without any stored value. This means that a variable must be assigned some value before we can refer to it in other statements. In the course code, we could not start code with the statements

```
var anInt in Int
anInt ← anInt + 122
```

The second statement needs some value to be stored in *anInt* to refer to when forming $anInt + 122$. In the course code, if you want *anInt* to start with the value 0, include an assignment setting it to this value:

```
var anInt in Int
anInt ← 0
```

Setting a starting value for a variable is referred to as **initialisation**.

Strictly *anInt* stores a value, say 122. This is described in various ways:

- (a) the state of *anInt* is 122;
 - (b) *anInt* takes the value 122;
 - (c) the value of *anInt* is 122;
- or even
- (d) *anInt* is 122.

All but the first of these abuse the language to some extent but are so common that we will accept their use.

In code, statements are executed in the order in which they are given (start at the top and work down to the bottom), and each new statement will start on a new line. (A long statement may run over more than one line.) The order in which assignments appear is important. For example, compare the effect of the following two fragments of code.

Fragment 1

```
var anInt in Int
anInt ← 1
anInt ← anInt + 122
anInt ← anInt × 2
```

Fragment 2

```
var anInt in Int
anInt ← 1
anInt ← anInt × 2
anInt ← anInt + 122
```

In Fragment 1, *anInt* takes the values, in turn: 1, 123, 246.

In Fragment 2, *anInt* takes the values, in turn: 1, 2, 124.

Reversing the order of the last two assignments changes the effect of the code.

Fragment 1 calculates $2 \times (1 + 122)$ whereas Fragment 2 calculates $(2 \times 1) + 122$.

Following the values taken by the variables helps us to see what a piece of code is doing. We refer to this activity as **tracing**. For code consisting of a sequence of assignments, we can set out a trace by writing values of each variable beside the statements. Below, we give an example of some code, together with a trace. In this trace, the values taken by the variables **after** each statement is executed are given to the right of the statement. (This layout will not work with more complicated code structures that you will meet later.)

	<i>varOne</i>	<i>varTwo</i>	<i>count</i>
var <i>varOne</i> in <i>Int</i>	-		
var <i>varTwo</i> in <i>Int</i>	-	-	
var <i>count</i> in <i>Int</i>	-	-	-
<i>varOne</i> ← 11	11	-	-
<i>varTwo</i> ← 3	11	3	-
<i>count</i> ← 0	11	3	0
<i>varOne</i> ← <i>varOne</i> − <i>varTwo</i>	8	3	0
<i>count</i> ← <i>count</i> + 1	8	3	1
<i>varOne</i> ← <i>varOne</i> − <i>varTwo</i>	5	3	1
<i>count</i> ← <i>count</i> + 1	5	3	2
<i>varOne</i> ← <i>varOne</i> − <i>varTwo</i>	2	3	2
<i>count</i> ← <i>count</i> + 1	2	3	3

We will not usually include variable declarations in a trace.

We put - in a trace where a variable has been created by a **var** statement, but has not yet been initialised.

Activity 1.1

Trace the values taken by the variables as the code below is executed.

```
var sum in Int
var number in Int
sum ← 0
number ← 0
number ← number + 1
sum ← sum + number
number ← number + 1
sum ← sum + number
number ← number + 1
sum ← sum + number
```

Discussion Below we give the value of the variables after each statement.

	<i>sum</i>	<i>number</i>
$sum \leftarrow 0$	0	-
$number \leftarrow 0$	0	0
$number \leftarrow number + 1$	0	1
$sum \leftarrow sum + number$	1	1
$number \leftarrow number + 1$	1	2
$sum \leftarrow sum + number$	3	2
$number \leftarrow number + 1$	3	3
$sum \leftarrow sum + number$	6	3

In traces given in the text, the code statements are repeated so that you can follow the solutions easily.

After this code has been executed, the variable *sum* stores the value 6, which is the result of adding together the values 1, 2 and 3 taken by the variable *number*. So this code leads to the value $1 + 2 + 3$ being stored in *sum*.

1.4 Comments

When writing code for a particular purpose, it can be helpful to include comments. In code, we show a **comment** by writing `/*` at the start of each comment and `*/` at the end of the comment. Comments are used to explain what code is doing. They are not addressed to the computer — which cannot interpret them and will just ignore them — but rather to a person trying to follow the code.

Activity 1.2

- (a) Trace the values taken by the variables as the code below is executed.

```

var sum in Int
var number in Int
var square in Int
sum  $\leftarrow 1$ 
number  $\leftarrow 1$ 
number  $\leftarrow number + 1$ 
square  $\leftarrow number \times number$ 
sum  $\leftarrow sum + square$ 

```

- (b) By modifying the code in Activity 1.1, write code after which the variable *sum* stores the quantity $1^2 + 2^2 + 3^2$.

Discussion

- (a) Values of the variables are traced below.

	<i>sum</i>	<i>number</i>	<i>square</i>
$sum \leftarrow 1$	1	-	-
$number \leftarrow 1$	1	1	-
$number \leftarrow number + 1$	1	2	-
$square \leftarrow number \times number$	1	2	4
$sum \leftarrow sum + square$	5	2	4

- (b) The code in Activity 1.1 leads to the value $1 + 2 + 3$ being stored in *sum*, where 1, 2 and 3 are the successive values taken by *number*. To store the required sum of squares in *sum*, we need to insert a statement to square the value in *number* each time, as we have in part (a) of this activity, and to add the resulting value (*square*, say) to *sum*.

The following code will do what is required. We have included comments in the code. These are not essential, but they do make the code easier for a person to interpret.

```

var sum in Int
var square in Int
var number in Int
sum  $\leftarrow$  0
number  $\leftarrow$  0
number  $\leftarrow$  number + 1
square  $\leftarrow$  number  $\times$  number
sum  $\leftarrow$  sum + square
/* sum now stores  $1^2$ . */
number  $\leftarrow$  number + 1
/* number now stores 2. */
square  $\leftarrow$  number  $\times$  number
sum  $\leftarrow$  sum + square
/* sum now stores  $1^2 + 2^2$ . */
number  $\leftarrow$  number + 1
/* number now stores 3. */
square  $\leftarrow$  number  $\times$  number
sum  $\leftarrow$  sum + square
/* sum now stores  $1^2 + 2^2 + 3^2$ . */

```

1.5 Calling functions in code

The code fragments above have involved integer variables, and binary operations on integers such as addition and subtraction. To handle other forms of data, code will need to call on appropriate processes. For example, if code is to handle characters, it will be helpful to be able to call on the functions *ASC* and *CHR*. Recall from Unit 1 that the function *ASC* inputs a character and returns its ASCII code, while the function *CHR* inputs a number between 32 and 126 and returns the associated character. An example of code calling on these functions is given below. On the right, the values taken by the variables *aChar* and *num* are traced.

ASCII codes are given in Table 2.1 of Unit 1, and are repeated in the *Course Handbook*.

```

var num in Int
var aChar in Char
aChar  $\leftarrow$  'q'
num  $\leftarrow$  ASC(aChar)
num  $\leftarrow$  num - 32
aChar  $\leftarrow$  CHR(num)

```

<i>num</i>	<i>aChar</i>
-	-
-	-
-	'q'
113	'q'
81	'q'
81	'Q'

To be able to trace this code, we needed to know the effect of the functions *ASC* and *CHR*. The descriptions of these functions given in Unit 1 provide enough information to enable this to be done. (Before a computer can actually execute a statement such as *num* $\leftarrow$ *ASC*(*aChar*), it will need some way of evaluating *ASC* for a given input. But we do not need to know the details of how this is done in order to trace code that uses *ASC*.)

Activity 1.3

Trace the values taken by the variables as the code above is executed, with the assignment $aChar \leftarrow 'q'$ replaced by $aChar \leftarrow 'b'$.

Discussion A trace is given below.

	<i>num</i>	<i>aChar</i>
$aChar \leftarrow 'b'$	-	'b'
$num \leftarrow ASC(aChar)$	98	'b'
$num \leftarrow num - 32$	66	'b'
$aChar \leftarrow CHR(num)$	66	'B'

1.6 Invalid code

If code includes statements that do not follow the rules of the course code, we will refer to that code as **invalid**. We will treat invalid code statements as if they do not work. A more precise interpretation would be to say that if code involves invalid statements, then we do not know what its effect will be.

Examples of invalid code would include:

1. using a variable that has not been created through a **var** statement;
2. attempting to store data of the wrong type in a variable;
3. using a variable, that has not been given an initial value, on the right-hand side of an assignment.

So, for example, each of the following code fragments is invalid.

```
var aChar in Char
aChar  $\leftarrow$  112
```

(an error of type 2 above), or

```
var aNum in Int
aNum  $\leftarrow$  aNum + 37
```

(an error of type 3).

We say that an assignment is **type consistent** if the data being stored is the same type as the variable.

1.7 Reserved words

You will notice that in the course code certain words, such as **var**, have been printed in bold. These are **reserved words** in the course code. Each reserved word has a special meaning, such as the creation of a new variable. These reserved words should not be used as the names of variables when writing your own code. (In particular, do not use *var* as a variable name!)

In this unit, we will use certain print conventions to emphasise the different elements in code statements. We will use italic for variables, and bold for reserved words. This formatting is not part of the course code. You do not need to use this formatting when writing code in assignments.

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: statement, literal, comment, assignment, initialisation, trace, invalid code, reserved word.
- ☐ Interpret and use the notation: **var** $\cdots$ **in** $\cdots$; $\cdots \leftarrow \cdots$.
- ☐ Trace a code fragment consisting of a sequence of assignments.
- ☐ Recognise that an assignment storing data of the incorrect type in a variable is invalid.

Exercises on Section 1

Exercise 1.1

Trace the values taken by the variables as the code below is executed.

```

var varOne in Int
var varTwo in Int
var count in Int
varOne  $\leftarrow$  14
varTwo  $\leftarrow$  5
count  $\leftarrow$  0
varOne  $\leftarrow$  varOne - varTwo
count  $\leftarrow$  count + 1
varOne  $\leftarrow$  varOne - varTwo
count  $\leftarrow$  count + 1

```

2 Conditionals

To be able to write code that performs various tasks, we need to add other forms of statement to the course code. In this section, we will consider **conditional** statements. An example of a conditional statement is given below. (The variable x , in the statement, is storing an integer.)

```
if ( $x <_{Int} 0$ ) then  $\{x \leftarrow 0\}$ 
```

If x stores a negative value when this statement is met, then the condition $x <_{Int} 0$ will be **true**, and the assignment $x \leftarrow 0$ is executed. But if x stores a positive value (or 0) when this statement is met, then $x \leftarrow 0$ is not executed, and the value stored in x will not be changed. Thus if x stores -5 before this statement, then x stores 0 after it. If x stores 3 before this statement, then it still stores 3 after it.

A statement of the general form

```
if (condition) then {code}
```

performs the code if the condition is **true**. If the condition is **false**, then the code is not performed. Sometimes we wish to perform some alternative action when the condition is **false**. To do this, we use a more general form of **conditional** statement, which includes an **else**. The effect of

if (condition) **then** {codeOne} **else** {codeTwo}

is to perform codeOne if the condition is **true**, and to perform codeTwo if the condition is **false**. For example, the statement

if ($x >_{Int} 0$) **then** $\{y \leftarrow 1\}$ **else** $\{y \leftarrow -1\}$

will set the variable y to 1 if x is positive, and will set the variable y to -1 if x is negative, or if x is 0.

Activity 2.1

Trace the values taken by the variables as the code below is executed.

```

var  $a$  in  $Int$ 
var  $b$  in  $Int$ 
var  $z$  in  $Int$ 
 $a \leftarrow -6$ 
 $b \leftarrow 3$ 
if ( $a <_{Int} 0$ ) then  $\{a \leftarrow (-1) \times a\}$ 
if ( $b <_{Int} 0$ ) then  $\{b \leftarrow (-1) \times b\}$ 
if ( $a <_{Int} b$ ) then  $\{z \leftarrow b\}$  else  $\{z \leftarrow a\}$ 

```

Discussion A trace is given below.

Code	Comment	a	b	z
$a \leftarrow -6$		-6	-	-
$b \leftarrow 3$		-6	3	-
if ($a <_{Int} 0$) $\{a \leftarrow (-1) \times a\}$	($a <_{Int} 0$) is true so use then	6	3	-
if ($b <_{Int} 0$)	($b <_{Int} 0$) is false no else , so no change	6	3	-
if ($a <_{Int} b$) $\{z \leftarrow a\}$	($a <_{Int} b$) is false , so use else	6	3	6

It may be that the action required when an **if** condition is **true** (or **false**) is more complicated than a single assignment. In such a case, the curly brackets, { and }, show the start and finish of the required actions. For example, consider Fragment 3 below.

Fragment 3

```

 $num \leftarrow ASC(aChar)$ 
if ( $(96 <_{Int} num) \wedge (num <_{Int} 123)$ ) then
{
     $num \leftarrow num - 32$ 
     $aChar \leftarrow CHR(num)$ 
}

```

Suppose that Fragment 3 is encountered with the variable $aChar$ in the state 'q'. The first assignment will set num to 113. Then $(96 <_{Int} num)$ will be **true**, and so will $(num <_{Int} 123)$. So the condition

$$((96 <_{Int} num) \wedge (num <_{Int} 123))$$

will be **true**. In this case, since the **if** condition is **true**, the bracketed statements after **then** are executed. So num is changed to $113 - 32 = 81$, and then $aChar$ is changed to 'Q'.

Fragment 3 is a portion taken from some larger piece of code. It will have been preceded at least by declarations **var** num **in** Int and **var** $aChar$ **in** $Char$, and an assignment initialising $aChar$.

Remember $\wedge$ is read "and".

Activity 2.2

The following questions all concern the code in Fragment 3 above.

- (a) Suppose Fragment 3 is encountered with *aChar* in the state '2'. What is the effect of Fragment 3 on the states of the variables *num* and *aChar*?
- (b) Suppose Fragment 3 is encountered with *aChar* in the state 'b'. What is the effect of Fragment 3 on the states of the variables *num* and *aChar*?
- (c) If the condition $(96 <_{Int} num) \wedge (num <_{Int} 123)$ is **true**, what can be said about the state of *aChar* before Fragment 3 is encountered?
- (d) Suppose Fragment 3 is encountered with *aChar* in an unknown (arbitrary) state. What can be said about the effect of Fragment 3 on the state of the variable *aChar*?

Discussion

- (a) If *aChar* is in the state '2' then $ASC(aChar) = 50$. So the first assignment sets *num* to 50. The condition $(96 <_{Int} num)$ is then **false**, so the condition following **if** is **false**. Consequently, the statements following **then** are not executed, and the code in Fragment 3 does not change the state of *aChar*.
- (b) If *aChar* is in the state 'b' then the first assignment sets *num* to $ASC('b')$, that is to 98. The condition $(96 <_{Int} num)$ is then **true**. The condition $(num <_{Int} 123)$ is also **true**. Thus the condition

$$(96 <_{Int} num) \wedge (num <_{Int} 123)$$

is **true**. Consequently, the statements following **then** are executed. So *num* is set to 66, and *aChar* is set to 'B'.

- (c) When Fragment 3 is encountered, *num* is set to the ASCII code of the character stored in *aChar*. The condition will be **true** if this character is a lower-case letter.
- (d) The **if** condition is **true** if *aChar* stores a lower-case letter when Fragment 3 is encountered. In this case, the statements following **then** are executed. The effect of these statements is to change the value stored in *aChar* to the corresponding upper-case letter. If *aChar* is not a lower-case letter, then the code in Fragment 3 does not change the state of *aChar*. So the effect of Fragment 3 is this.

If *aChar* is a lower-case letter when Fragment 3 is encountered then *aChar* is changed to the corresponding letter in upper case. Otherwise, *aChar* is left unchanged.

Sometimes code may nest a conditional statement inside another conditional. As an example, consider Fragment 4 below.

Fragment 4

```

num ← ASC(aChar)
if (96 <Int num) then
{
    if (num <Int 123) then
    {
        num ← num − 32
        aChar ← CHR(num)
    }
}

```

Suppose that Fragment 4 is encountered with the variable *aChar* in the state ‘q’. Then *num* will be set to 113. The condition $(96 <_{Int} num)$ will be **true**, so the statements after the first **then** are executed. These statements consist of the inner conditional statement. The **if** condition, $(num <_{Int} 123)$, in that inner conditional is also **true**. So the bracketed statements after the second **then** are executed. Thus *num* is changed to 81, and then *aChar* is changed to ‘Q’.

You may have noticed that the effect of Fragment 4 is the same as that of Fragment 3. The statements after the second **then** in Fragment 4 are executed when both the **if** conditions are **true**, and so this code has the identical effect to Fragment 3.

Activity 2.3

For the code given below, there are four possible actions which may occur: (a) $q \leftarrow 1$; (b) $q \leftarrow 2$; (c) $q \leftarrow 3$; (d) $q \leftarrow 4$. For which values of the variables *x* and *y* (both in *Int*) will each of these possible actions occur?

```

if ( $x >_{Int} 0$ ) then
{
    if ( $y >_{Int} 0$ ) then  $\{q \leftarrow 1\}$  else  $\{q \leftarrow 2\}$ 
}
else
{
    if ( $y >_{Int} 0$ ) then  $\{q \leftarrow 3\}$  else  $\{q \leftarrow 4\}$ 
}

```

Discussion The statement

if ($y >_{Int} 0$) **then** $\{q \leftarrow 1\}$ **else** $\{q \leftarrow 2\}$

is executed if $x >_{Int} 0$ is **true**, while the statement

if ($y >_{Int} 0$) **then** $\{q \leftarrow 3\}$ **else** $\{q \leftarrow 4\}$

is executed if $x >_{Int} 0$ is **false**. Thus:

- (a) $q \leftarrow 1$ occurs if $(x >_{Int} 0)$ and $(y >_{Int} 0)$ are both **true**.
- (b) $q \leftarrow 2$ occurs if $(x >_{Int} 0)$ is **true** and $(y >_{Int} 0)$ is **false**; i.e. if $(x >_{Int} 0)$ and $(y \leq_{Int} 0)$.
- (c) $q \leftarrow 3$ occurs if $(x >_{Int} 0)$ is **false** and $(y >_{Int} 0)$ is **true**; i.e. if $(x \leq_{Int} 0)$ and $(y >_{Int} 0)$.
- (d) $q \leftarrow 4$ occurs if $(x >_{Int} 0)$ is **false** and $(y >_{Int} 0)$ is **false**; i.e. if $(x \leq_{Int} 0)$ and $(y \leq_{Int} 0)$.

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: conditional.
- ☐ Interpret and use the notation: **if** ($\dots$) **then** $\{\dots\}$ **else** $\{\dots\}$.
- ☐ For a given initial state, trace a code fragment including conditional statements.

Exercises on Section 2

Exercise 2.1

Trace the code fragment below if it is encountered with n in the state: (a) 16; (b) 24. ($aBool$ is a variable of type *Bool* and n is a variable of type *Int*.)

```

aBool ← true
if (MOD(n, 2) =Int 0) then {n ← DIV(n, 2)} else {aBool ← false}
if (MOD(n, 3) =Int 0) then {n ← DIV(n, 3)} else {aBool ← false}
if (MOD(n, 2) =Int 0) then {n ← DIV(n, 2)} else {aBool ← false}

```

Recall that *DIV* is integer division and *MOD* gives the remainder from integer division so that, for example, $DIV(7, 3) = 2$ and $MOD(7, 3) = 1$.

Exercise 2.2

- (a) Trace the code in Fragment 5 below if it is encountered with c in the state: (i) 'J'; (ii) 'j', (iii) '7'.
- (b) Suppose that Fragment 5 is encountered with c in an unknown (arbitrary) state. What can be said about the effect of Fragment 5 on the state of the variable $aBool$?

Fragment 5

```

aBool ← false
n ← ASC(c)
if (64 <Int n) then
{
  if (n <Int 91) then {aBool ← true}
}
if (96 <Int n) then
{
  if (n <Int 123) then {aBool ← true}
}

```

Code fragments will not always contain variable declarations. The type of such variables will be clear from the context. For example, in this fragment it is clear that $aBool$ is a Boolean variable and that n is in *Int*.

Exercise 2.3

Write code that has the following effect on a variable $aChar$ from *Char*.

If $aChar$ is an upper-case letter then $aChar$ is changed to the corresponding letter in lower case. Otherwise, $aChar$ is left unchanged.

3 Specification

In Sections 1 and 2, you have seen code that called on processes such as addition of numbers and the function *ASC*. You can follow the effect of code that calls on a function such as *ASC* by referring to a description of that function. If such code is to be executed by a computer, then the computer needs to be able to find the values returned by these processes. However, to determine the effect of code calling on *ASC*, say, we do not need to know how the computer calculates the values returned by *ASC*.

In this section we shall not be concerned with how a process that is called in a statement may be evaluated by a computer. Here, we keep to an outside view of a process. We are concerned only to know what a computing process does, so that we can predict how code calling on that process will behave.

We refer to a description of a computing process, as seen from the outside, as a **specification** of that process.

We shall assume that certain “primitive” processes, such as addition of numbers, the function *ASC*, the binary operation $\wedge$, and the comparison $<_{Int}$ are available to use in code, and we shall not discuss how these are implemented by a computer. These primitive processes do not, for example, include one that changes a lower-case letter to the corresponding letter in upper case.

In Unit 3, we will introduce the idea of data types, and it will be clear precisely which processes are available to be used in code.

3.1 Specifying a function

All the processes that we will be concerned with in this unit take certain inputs and return some value as an output. Any process of this form corresponds directly to a mathematical function. We will use the same layout to give a specification of a function available in code, as we used to describe a mathematical function in Unit 1. The only difference is that we will add a heading **function** to the description. As an example, a specification of a function *LAST* is given below.

function *LAST*(*s* in *SeqOfX*) **return** in *X*

pre The sequence *s* has at least one member.

post The returned value is the last member of *s*. (So, for example, if $s = [1, 2, 3, 4]$, then the returned value is 4.)

Underlying such a specification is some code, that the computer can interpret, which enables the computer to find the value *LAST*(*s*) when a specific input sequence *s* is supplied. You do not need to see this code in order to use the function *LAST* in your own code. You do, though, need to be confident that the code underlying *LAST* works. The specification can be seen as a contract between the person who wrote that underlying code and anyone who uses it.

The first line of the specification is called the signature line. Once data types have been introduced in Unit 3, the signature line will be given in terms of data types rather than using sets, as here.

A user must only call *LAST* with an appropriate input. To recognise an appropriate input, you look at two things. The first line of the specification shows how many inputs *LAST* requires, and what form of data they must have. The second line of the specification gives the **precondition**. This gives any additional conditions that the input(s) must satisfy. It is your responsibility as a user to make sure that you give *LAST* the correct number of inputs, of the correct type, and in the appropriate order, and that these inputs satisfy the precondition. If you do that, then the output will satisfy the **postcondition**, given in the final line.

If no additional conditions are required the precondition is given as **true**.

It is the responsibility of whoever wrote the code implementing *LAST* to ensure that the value returned by the code satisfies the postcondition. As a user of *LAST*, you do not need to see that code, nor worry about whether it works correctly. You should be able to rely on the other person in the contract to have done their bit.

When calling a process in your own code, you must be sure to use it in accordance with its specification. If you do not, we shall regard the code as invalid. For example the code

```
var s in SeqOfChar
var t in Char
s ← [ ]
t ← LAST(s)
```

is invalid as *s* does not satisfy the precondition of the function *LAST*.

A function may require more than one input, as does *ISINSTR*, whose specification is given below.

function *ISINSTR*(*c* in *Char*, *s* in *SeqOfChar*) **return** in *Bool*

pre true.

post The returned value is **true** if the character *c* appears in the string *s* (one or more times). It is **false** if *c* does not appear in *s*, or if *s* is empty.

In the course code, when calling such a function, the inputs must be given in the correct order, as shown in the specification. So *ISINSTR*('a', "this") is a valid call, but *ISINSTR*("this", 'a') is not valid. Taking care about the order of the inputs is particularly important when there are two inputs of the same type, and code is valid for either order of these inputs, but a different value is returned in the two cases. (For example, *MOD*(13, 5) is 3, but *MOD*(5, 13) is 5.)

Recall that we use double inverted commas as a compact notation for a sequence of characters (or string). So, for example, "this" stands for ['t', 'h', 'i', 's'].

Activity 3.1

- (a) The process *ISINSTR* is specified above. Write down the signature of the corresponding mathematical function.
- (b) The function *REMOVEDLAST* has signature

$$REMOVEDLAST : SeqOfX \rightarrow SeqOfX$$

and, for a non-empty input sequence, returns that sequence with its last member removed. (So, for example, *REMOVEDLAST* ([1,2,3,4]) = [1,2,3].) Give a specification of *REMOVEDLAST*.

Discussion

- (a) Here we are specifically asked for the signature of the mathematical function so we should use the notation introduced in Unit 1. We have to use the Cartesian product $Char \times SeqOfChar$ for the input set since there are two input values, a character and a sequence of characters (in that order). The output set is *Bool* and so the signature is

$$ISINSTR : Char \times SeqOfChar \rightarrow Bool.$$

- (b) A specification is given below.

function *REMOVEDLAST*(*s* in *SeqOfX*) **return** in *SeqOfX*

pre The sequence *s* has at least one member.

post The returned value is formed by removing the last member of *s*. (So, for example, if *s* = [1, 2, 3, 4], then the returned value is [1,2,3].)

The description of the function indicated that the input sequence should be non-empty. We have shown this in the precondition.

Activity 3.2

Trace the values of the variables in the code in Fragment 6 below. Specifications of *LAST* and *REMOVELAST* are given above.

Fragment 6

```

var sum in Int
var aSeq in SeqOfInt
aSeq  $\leftarrow$  [3,6,1,9]
sum  $\leftarrow$  0
sum  $\leftarrow$  sum + LAST(aSeq)
aSeq  $\leftarrow$  REMOVELAST(aSeq)
sum  $\leftarrow$  sum + LAST(aSeq)
aSeq  $\leftarrow$  REMOVELAST(aSeq)
sum  $\leftarrow$  sum + LAST(aSeq)
aSeq  $\leftarrow$  REMOVELAST(aSeq)
sum  $\leftarrow$  sum + LAST(aSeq)
aSeq  $\leftarrow$  REMOVELAST(aSeq)

```

Discussion A trace is given below.

	<i>sum</i>	<i>aSeq</i>
<i>aSeq</i> $\leftarrow$ [3,6,1,9]	-	[3,6,1,9]
<i>sum</i> $\leftarrow$ 0	0	[3,6,1,9]
<i>sum</i> $\leftarrow$ <i>sum</i> + <i>LAST</i> (<i>aSeq</i>)	9	[3,6,1,9]
<i>aSeq</i> $\leftarrow$ <i>REMOVELAST</i> (<i>aSeq</i>)	9	[3,6,1]
<i>sum</i> $\leftarrow$ <i>sum</i> + <i>LAST</i> (<i>aSeq</i>)	10	[3,6,1]
<i>aSeq</i> $\leftarrow$ <i>REMOVELAST</i> (<i>aSeq</i>)	10	[3,6]
<i>sum</i> $\leftarrow$ <i>sum</i> + <i>LAST</i> (<i>aSeq</i>)	16	[3,6]
<i>aSeq</i> $\leftarrow$ <i>REMOVELAST</i> (<i>aSeq</i>)	16	[3]
<i>sum</i> $\leftarrow$ <i>sum</i> + <i>LAST</i> (<i>aSeq</i>)	19	[3]
<i>aSeq</i> $\leftarrow$ <i>REMOVELAST</i> (<i>aSeq</i>)	19	[]

We will use the functions *LAST* and *REMOVELAST* in a number of examples later in the unit. We will not repeat the specifications each time.

Activity 3.3

- Say what form of data each of the variables in Fragment 7 below must have.
- Describe the effect of Fragment 7 on each of the variables *aSeq* and *bSeq*.
- Under what circumstances is *b* **true** after Fragment 7 has been executed?

Fragment 7

```

b  $\leftarrow$  true
if (NOT(LAST(aSeq) =Int LAST(bSeq))) then {b  $\leftarrow$  false}
aSeq  $\leftarrow$  REMOVELAST(aSeq)
bSeq  $\leftarrow$  REMOVELAST(bSeq)

```

Discussion

- (a) To be a suitable input to $LAST$, $aSeq$ must be a sequence. To be a suitable input to $=_{Int}$, the value $LAST(aSeq)$ must be an integer. So $aSeq$ must be from $SeqOfInt$. (You may also have noted that to be a suitable input to $LAST$, $aSeq$ must satisfy the precondition of $LAST$, and so must be a non-empty sequence.) For the same reasons, $bSeq$ must also be from $SeqOfInt$ (and must be a non-empty sequence). The value of the variable b must be from $Bool$.
- (b) The only statement changing the state of $aSeq$ is
- $$aSeq \leftarrow REMOVELAST(aSeq)$$
- This removes the last integer from the sequence $aSeq$. Similarly, the effect of the fragment on $bSeq$ is to remove its last integer.
- (c) If, when the fragment is encountered, the last integer in $aSeq$ is the same as the last integer in $bSeq$, then, after the fragment is executed, b is **true**, while if these integers are different, b is **false**.
-

You saw in Unit 1 that we use infix notation for binary operations such as addition of integers, and for comparisons such as $<_{Char}$. It is important that it is made clear when infix notation is to be used so that code can be correctly interpreted. A process that is called in code using infix notation is flagged in its specification by including (**infix**) after **function**. For example, a specification of $<_{Char}$ is given below.

function (**infix**) ($c <_{Char} d$ **on** $Char$) **return in** $Bool$

pre **true**.

post The returned value is **true** if $ASC(c) <_{Int} ASC(d)$ and is **false** otherwise.

In the course code, an infix function will always be called using infix notation. So we will write ' a ' $<_{Char}$ ' g ' (and not $<_{Char}('a', 'g')$), for example.

Activity 3.4

Give specifications of the following computing processes.

- (a) CHR , which returns the character with a given ASCII value.
- (b) A binary operation $=_{Str}$, using infix notation, which compares two strings and returns **true** if they are the same.

Discussion Specifications are given below. For (b), recall from Unit 1 that a string is a sequence of characters, and so comes from the set $SeqOfChar$.

- (a) **function** $CHR(n$ **in** $Int)$ **return in** $Char$

pre $(31 <_{Int} n)$ and $(n <_{Int} 127)$.

post The returned value is the character with ASCII code n .

- (b) **function** (**infix**) ($s =_{Str} t$ **on** $SeqOfChar$) **return in** $Bool$

pre **true**.

post The returned value is **true** if s and t contain the same characters in the same order, or if s and t are both empty, and is **false** otherwise.

3.2 Specifying a user-defined process

Activity 3.5

- (a) Modify the code in Fragment 6 in Activity 3.2 by replacing the assignment $aSeq \leftarrow [3,6,1,9]$ with $aSeq \leftarrow [4,7,2,10]$, and trace the modified code.
- (b) Look back to Activity 3.2. Can you see a relationship between the value stored in sum after the code in Fragment 6 has been executed and the value $[3,6,1,9]$ initially assigned to $aSeq$? Does this relationship still hold when the value initially assigned to $aSeq$ is modified as in (a) above?

Discussion

- (a) A trace is given below.

	sum	$aSeq$
$aSeq \leftarrow [4,7,2,10]$	-	$[4,7,2,10]$
$sum \leftarrow 0$	0	$[4,7,2,10]$
$sum \leftarrow sum + LAST(aSeq)$	10	$[4,7,2,10]$
$aSeq \leftarrow REMOVELAST(aSeq)$	10	$[4,7,2]$
$sum \leftarrow sum + LAST(aSeq)$	12	$[4,7,2]$
$aSeq \leftarrow REMOVELAST(aSeq)$	12	$[4,7]$
$sum \leftarrow sum + LAST(aSeq)$	19	$[4,7]$
$aSeq \leftarrow REMOVELAST(aSeq)$	19	$[4]$
$sum \leftarrow sum + LAST(aSeq)$	23	$[4]$
$aSeq \leftarrow REMOVELAST(aSeq)$	23	$[]$

- (b) In Activity 3.2, the value stored in sum after Fragment 6 has been executed is 19. This is the sum of the numbers in the sequence $[3,6,1,9]$. That is,

$$19 = 3 + 6 + 1 + 9.$$

In the trace in (a), the value stored in sum after the modified code has been executed is 23. This is again the sum of the numbers in the sequence $[4,7,2,10]$. That is,

$$23 = 4 + 7 + 2 + 10.$$

The code in Fragment 6 adds up the numbers in the sequence of numbers $[3,6,1,9]$, which is the value initially assigned to the variable $aSeq$. Aside from that initial assignment, the code would do exactly the same job for any sequence of four numbers. You will see in Section 4 how to capture this generality in the course code.

Here, we want to give an external view of the effect of such generalised code. A suitable function, called *ADDFOUR*, is specified below.

function *ADDFOUR*(s in *SeqOfInt*) **return** in *Int*

pre The sequence s contains exactly four members.

post The returned value is the sum of the four numbers in s .

A specification provides a way of documenting a process that is available in code. To state the effect of some piece of code that you will write yourself, (or, indeed, that you might ask someone else to write for you) a specification again can be used.

Activity 3.6

Specify a function *COUNTFIVE* that searches a given string containing exactly five characters to determine how often a given character appears in the string. (So, for example, *COUNTFIVE*('b', "babel") = 2.)

Discussion The function has two inputs: a string (sequence of characters) and the character being looked for. It will output an integer. The semantics need to deal with the case when the given character does not appear at all in the string. The natural output in that case is 0. A suitable specification is given below.

function *COUNTFIVE*(*c* in *Char*, *s* in *SeqOfChar*) **return** in *Int*

pre The sequence *s* contains exactly five members.

post The returned value is the number of times that *c* appears in *s*. The returned value is 0 if *c* does not appear in *s*.

Recall from Unit 1 that the semantics of a function are given in the postcondition and show the relationship between input and output.

3.3 Checking that function calls are valid

When a function is called, the inputs must be of the correct type (in the correct order). Also, if a function has a precondition, the function call must involve inputs that satisfy that precondition. It may be difficult to see whether this is the case.

Activity 3.7

Explain why each of the code fragments in (a)–(c) is not valid.

- (a) **var** *num* **in** *Int*
var *aChar* **in** *Char*
aChar $\leftarrow$ '&'
num $\leftarrow$ *ASC*(*aChar*)
num $\leftarrow$ *num* – 32
aChar $\leftarrow$ *CHR*(*num*)
- (b) **var** *aChar* **in** *Char*
var *aVar* **in** *Int*
var *varTwo* **in** *Int*
aChar $\leftarrow$ 'Q'
aVar $\leftarrow$ 5
varTwo $\leftarrow$ *DIV*(*aChar*, *aVar*)
- (c) **var** *numOne* **in** *Int*
var *aSeq* **in** *SeqOfInt*
aSeq $\leftarrow$ [22,86]
numOne $\leftarrow$ *LAST*(*aSeq*)
aSeq $\leftarrow$ *REMOVELAST*(*numOne*)

Discussion

- (a) Since $ASC('&') = 38$, num is set to 38 (by $num \leftarrow ASC(aChar)$), and then to 6 (by $num \leftarrow num - 32$). The next statement tries to find $CHR(6)$. However 6 does not satisfy the precondition of CHR , so this statement is not valid.
- (b) In the last statement, the first value input to DIV is of the wrong type. DIV requires two inputs that are integers; but $aChar$ stores a character.
- (c) $numOne$ is a variable of type Int . So the assignment

$aSeq \leftarrow REMOVELAST(numOne)$

calls the function $REMOVELAST$ with an input that is an integer. But $REMOVELAST$ requires a sequence as an input (as is shown by the signature line in the specification). So $REMOVELAST(numOne)$ calls $REMOVELAST$ with an input of the wrong type, and this statement is invalid.

Activity 3.8

In Fragment 8 below, assume that c is a character for which we know that $ASC(c) > 63$.

- (a) Add comments to the code in Fragment 8 below to show why you can be sure that the function call $CHR(num)$ is valid; that is, that the value stored in num when the call is made satisfies the precondition of CHR .
- (b) What is the relationship between the character value c passed to the code in Fragment 8 and the value stored in $aChar$ after that code has been executed?

Fragment 8

```

var num in Int
var aChar in Char
aChar  $\leftarrow$  c
num  $\leftarrow$  ASC(aChar)
num  $\leftarrow$  num - 32
aChar  $\leftarrow$  CHR(num)

```

Discussion

- (a) Appropriate comments have been inserted below.

```

var num in Int
var aChar in Char
aChar  $\leftarrow$  c
/* The given condition on c ensures that ASC(aChar) > 63. */
num  $\leftarrow$  ASC(aChar)
/* Now num > 63 and since num is a value returned by ASC,
we also know that num < 127. */
num  $\leftarrow$  num - 32
/* Now num > 31 (and num < 95) and so num satisfies
the precondition of CHR. */
aChar  $\leftarrow$  CHR(num)

```

- (b) You saw two examples of the effect of this code in Section 1. If c is 'q', then $aChar$ is 'Q' at the end. If c is 'b', then $aChar$ is 'B' at the end. This suggests that the effect of Fragment 8 is to change a lower-case character into its upper-case equivalent. And that is just what the code does, so long as c is a lower-case letter.

However, the condition given on the value c passed to Fragment 8 does not force c to be a lower-case letter. That condition only requires c to have an ASCII code of at least 64. For a general character input, the final value stored in $aChar$ after Fragment 8 is executed is a character whose ASCII code is 32 less than the ASCII code of c . Subtracting 32 from the ASCII code has the effect of changing a lower-case letter to its upper-case equivalent, if the character c that we start with happens to be a lower-case letter.

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Interpret and use the notation for specifying a computing process.
- ☐ Trace a code fragment involving calls of computing processes for which a specification is given.
- ☐ Recognise function calls that are not valid, either because they have an input of the wrong type, or because an input fails to satisfy the precondition.

Exercises on Section 3

Exercise 3.1

Give a specification in the course code for each of (a)–(c).

- (a) A function $ADDLAST$, which adds an item from the set X at the end of a sequence of items from X (so $ADDLAST(5, [-3, 2, 4]) = [-3, 2, 4, 5]$, for example).
- (b) $INTHREE$, which searches a string containing exactly three characters to determine whether or not a given character appears in the string.
- (c) A comparison $=_{Char}$, using infix notation, which determines whether two characters are identical.

Exercise 3.2

Explain why the code below is invalid.

```
var aSeq in SeqOfChar
aSeq ← "abC"
aSeq ← REMOVELAST(aSeq)
aSeq ← REMOVELAST(aSeq)
aSeq ← REMOVELAST(aSeq)
aSeq ← REMOVELAST(aSeq)
```

4 Implementation

In this section, we will look at code to evaluate a process for which a specification has been given.

4.1 Implementing a user-defined function

Suppose that we want to make the function specified below available in code.

function *MAKEUPPER*(*c in Char*) **return in Char**

pre The character *c* is a lower-case letter.

post The returned value is the letter *c* in upper case. So, for example, if *c* = 'q' then the returned value is 'Q'.

Fragment 8 in Section 3 gives code that can be used to find the values returned by this process. In the course code, we can package that code as shown below. The specification appears as a **header** at the start of the code. This provides a summary of the code: the inputs it requires, and the value it returns. Packaged like this, the code is referred to as an implementation of the specification. The start and finish of the statements forming the implementation are shown by curly brackets: { and }. The final statement shows the value returned by the packaged code. In this case, this is the final value of the variable *aChar*. Occasionally, we may need to refer to an implementation by name. If so, that can be included as a comment, as we have done for the reference name *impMakeUpper* in the code below.

function *MAKEUPPER*(*c in Char*) **return in Char**

pre The character *c* is a lower-case letter.

post The returned value is the letter *c* in upper case. So, for example, if *c* = 'q' then the returned value is 'Q'.

```
/* impMakeUpper */
{
  var num in Int
  var aChar in Char
  aChar ← c
  num ← ASC(aChar)
  num ← num - 32
  aChar ← CHR(num)
  return aChar
}
```

In the assignment *aChar* ← *c* here, *c* may be any value that is consistent with the specification. The type of *c* is given by the signature line of *MAKEUPPER* and shows that *c* must be a character. Also, *c* must satisfy the precondition in the specification (which in this case states that *c* is a lower-case letter).

The specification is a declaration of what the code does. If suitable inputs are supplied, then the code will return a value, and this value will satisfy the postcondition. If you write code to implement some specification, you need to be sure that the code does what is claimed in the specification! An implementation of a specification is said to be **correct**, if it really does what is claimed in the specification. So if you claim that code is correct, you are saying that:

if the code is supplied with input(s) of the correct type(s), and satisfying the precondition, then the code is valid, and will return a value satisfying the postcondition.

When doing exercises, you can assume that any process that we say is available in code is correctly implemented. But if we ask you to give an implementation of some specification, then a key part of your task is to make sure that it is correct!

Packaging code into a function has a number of advantages. Suppose that you have written code to perform a particular action, then find that you need to perform the same action in another context. You could just copy the code. However if you use a specification to make precise the role of the code you already have, then this will help you to verify that it can really be transferred safely to the new context. For example, part of that verification is to check whether the function will be called with input values that satisfy the precondition. If your existing code is a correct implementation of the specification and you have carried out such a verification, then it is certainly still correct in the new context! And keying in a single function call requires less effort and holds less risk of a mechanical keying error than does keying in a copy of the code.

In general, if you are writing code that you may want to reuse, or that you want other people to be able to use, then you should provide a specification for the code. The systematic approach is first to supply the specification, then write code to implement that specification. In practice, things will not always be so clear, since at first you may not be sure exactly what specification you want. But attaching a specification to code that you have written still provides important documentation. Suppose you are writing code to change the case of a letter character from lower to upper case. Your first thought may be that the code packaged in *impMakeUpper* is sufficient for your purpose. Then later, you may find that you need other characters to be left unchanged. If you have provided a specification of *MAKEUPPER*, then that will immediately remind you that the code in *impMakeUpper* is not adequate for your refined purpose, and that you need to provide a new specification, and an implementation of that. (See the function *MAKEUPPERTWO*, discussed later in this section.)

Activity 4.1

In Section 3 you saw the specification of a function *ADDFOUR*, and code suitable to implement that function. Write out an implementation of *ADDFOUR* using the course notation.

Discussion This is mostly a matter of putting together material from Section 3. You can use the code in Fragment 6 (Activity 3.2), modified to show a general sequence *s* in the first assignment, and adding a **return** statement at the end of the code.

function *ADDFOUR*(*s* in *SeqOfInt*) **return** in *Int*
pre The sequence *s* contains exactly four members.
post The returned value is the sum of the four numbers in *s*.

```
/* impAddFour */
{
  var sum in Int
  var aSeq in SeqOfInt
  aSeq ← s
```

```

sum ← 0
sum ← sum + LAST(aSeq)
aSeq ← REMOVELAST(aSeq)
sum ← sum + LAST(aSeq)
aSeq ← REMOVELAST(aSeq)
sum ← sum + LAST(aSeq)
aSeq ← REMOVELAST(aSeq)
sum ← sum + LAST(aSeq)
aSeq ← REMOVELAST(aSeq)
return sum
}

```

Variables that are declared within an implementation of a function are said to be **local** to that implementation. In *impAddFour*, *sum* and *aSeq* are local variables. The association between each of the local variables *sum* and *aSeq* and a storage area in computer memory applies only while the implementation *impAddFour* is being executed. These local variables cannot be referred to in outside code that calls the function *ADDFOUR*.

In the discussion of Activity 4.1, *s* is used as the identifier for a value passed to the code in *impAddFour*. When *ADDFOUR* is called in outside code, some value stored in that outside code is passed to *ADDFOUR* as an input. It is important that execution of the code in *impAddFour* will not change this value stored in the outside code. For this reason, we would not write an instruction such as *s* ← *REMOVEDLAST*(*s*) in *impAddFour*.

In the course code, the specification of any function includes a signature line that gives identifiers for the inputs, as in the line

function *ADDFOUR*(*s* **in** *SeqOfInt*) **return in** *Int*,

which gives *s* as the identifier of the sequence that is input to *ADDFOUR*. In code to implement this function, *s* cannot be used as a local variable. (That is, you could not have a **var** statement to declare *s* as a variable within the implementation of *ADDFOUR*.) This is a general point to note. In the course code, any instruction that attempts to change a value stored in a variable identifying an input to a function is invalid.

This is important! Do not use the identifiers of input variables as identifiers of local variables in an implementation of a function.

Doing this also means that the input values are always available to you when you are writing code. Within code implementing *ADDFOUR*, say, any use of *s* would be a reference to the value being input to the function. No intervening statements could have changed the value stored by *s*.

One could equally well use some other identifier for the input to *ADDFOUR*, such as *t*. Each occurrence of *s* in *ADDFOUR* and *impAddFour* could be replaced by *t*, without changing the meaning of the code. The variable *s* is not a reference to a specific storage location in the computer. It is a placeholder, representing a value that will be found when the function is called in some specific context.

Another point to note is that the identifier *s* used for the input to *ADDFOUR* is **local** to the specification and implementation given above. The meaning of *s* as the value of the input to *ADDFOUR* applies from the introduction of *s* in the line

function *ADDFOUR*(*s* **in** *SeqOfInt*) **return in** *Int*,

to the **}** at the end of *impAddFour*. Outside of that context, *s* can be used to mean something else.

In *impAddFour*, suppose the assignment $aSeq \leftarrow REMOVELAST(aSeq)$ just before **return** *sum* were to be deleted. This assignment has no effect on the final value of *sum*, so does not change the value that is returned. Thus we would still have code that correctly implements *ADDFOUR*. But it is different code. For a given specification, different implementations may be possible.

If code is sufficiently simple, then one may be able to trace it for a general input value. For example, consider the code below.

```

var num in Int
num  $\leftarrow x$ 
num  $\leftarrow num + 1$ 
num  $\leftarrow num \times num$ 

```

The values taken by *num* are, in turn: x , then $x + 1$, then $(x + 1) \times (x + 1)$. This code is valid for any integer x . (x must be an integer to make the assignment $num \leftarrow x$ type consistent.) Since $(x + 1) \times (x + 1)$ is the same as $(x + 1)^2$, this code can be used to implement the function below.

```

function ADDSQ(x in Int) return in Int
pre   true.
post  The returned value is  $(x + 1)^2$ .

```

Activity 4.2

- (a) Trace the code below for a general input value x . Give a formula for the value returned by this code.

```

var num in Int
var number in Int
num  $\leftarrow x$ 
number  $\leftarrow num \times 2$ 
number  $\leftarrow number + 1$ 
num  $\leftarrow num \times num$ 
number  $\leftarrow number + num$ 
return number

```

- (b) Specify a function that the code in (a) could implement.

Discussion

- (a) A trace is given below. The value returned is $x^2 + 2x + 1$.

	<i>num</i>	<i>number</i>
$num \leftarrow x$	x	-
$number \leftarrow num \times 2$	x	$2x$
$number \leftarrow number + 1$	x	$2x + 1$
$num \leftarrow num \times num$	x^2	$2x + 1$
$number \leftarrow number + num$	x^2	$x^2 + 2x + 1$

- (b) Since the variable *num* is an integer, x must be an integer. The value returned is the value of *number* and so is also an integer. There is no need for a precondition as the statements (and the formula for the returned value) are valid for any integer x . A suitable specification is given below.

```

function ADDSQTWO(x in Int) return in Int
pre   true.
post  The returned value is  $x^2 + 2x + 1$ .

```

Now multiplying out the brackets will show that $(x + 1) \times (x + 1)$ is equal to $x^2 + 2x + 1$. So this function *ADDSQ TWO* is actually identical to the function *ADDSQ*, whose specification is given just before this activity. This is another example of two different implementations of the same specification.

When writing code to implement some given specification you need to work with processes that are already available. This raises the question “What processes are available?” We will address this in a systematic way in Units 3 and 4. In the remainder of this unit, we will list the processes available for you to work with on an ad hoc basis.

A specification and the code implementing it are closely related. The code must correctly implement the specification! However, it is important to keep the specification separate from the code that implements it. Maintenance of such separation is referred to as **abstraction**. To see the effect of a process you need a clear and precise specification to refer to. In addition, specifying the desired effect of code gives a criterion for checking whether code is performing its intended purpose.

Abstraction is a key tool in situations where complex code is being produced. Psychological experiments suggest that people can comprehend between five and nine pieces of information at any one moment. Packaging code into functions, and ensuring that any one function only requires a few lines of code to implement it, provides a way of controlling complexity. Using a specification to state the effect of the code (rather than looking at the details of the code itself) is a key part of this technique. If a complex task is to be divided among a team, then a systematic division of the task into parts is essential. Specifying each part helps to keep track of the divided task. (The parts might be functions, but may have some more general form.)

See for example: Miller G A
“The magic number seven,
plus or minus two: limits on
our capacity for processing
information”. *Psychological
Review* **63**(2) p 86 (1956).

Activity 4.3

The function *CEREALPRICE* inputs a character (representing the size of a packet of cereal), that may be ‘S’ (for Small), ‘M’ (for Medium) or ‘L’ (for Large) and returns the associated price as an integer number of pence. The price of a packet of cereal (in pence) is: 89 for Small, 119 for Medium and 159 for Large. Give a specification and implementation of the function *CEREALPRICE*. Your code may use the infix binary operation: $=_{Char}$, which returns **true** if two characters are the same and **false** otherwise.

Discussion The specification and implementation are given below.

function *CEREALPRICE*(*size* in *Char*) **return** in *Int*

pre *size* is in the set {‘S’, ‘M’, ‘L’}.

post The returned value gives the price in pence of a packet of cereal of the specified size.

```
{
  var cost in Int
  if (size =Char ‘S’) then {cost ← 89}
  if (size =Char ‘M’) then {cost ← 119}
  if (size =Char ‘L’) then {cost ← 159}
  return cost
}
```

4.2 Calling a user-defined function in code

Recall that the function *MAKEUPPER* changes the case of a lower-case letter, and has a precondition that excludes other inputs. Suppose that you have already written an implementation of *MAKEUPPER*, and that you now want a function *MAKEUPPERTWO*, which changes the case of a lower-case letter and leaves other characters unchanged. To reuse the code you have already written, you can call the function *MAKEUPPER* in an implementation of *MAKEUPPERTWO*, as below.

```
function MAKEUPPERTWO(d in Char) return in Char
pre   true.
post  If d is a lower-case letter, then the returned value is the letter d in
upper case. If d is not a lower-case letter, then the returned value is d.

/* impMakeUpperTwo */
{
  var theChar in Char
  theChar  $\leftarrow$  d
  if ((96 <Int ASC(theChar))  $\wedge$  (ASC(theChar) <Int 123)) then
    {theChar  $\leftarrow$  MAKEUPPER(theChar)}
  return theChar
}
```

You have seen code that implements *MAKEUPPER*, since we gave an implementation, *impMakeUpper*, of this function earlier. However, the rest of the code (referred to later as the outside code) in *impMakeUpperTwo* is completely insulated from what happens inside the implementation of *MAKEUPPER*. The variables *num* and *aChar* declared inside *impMakeUpper* are **local** to that implementation. They are not accessible in code outside *impMakeUpper*.

We introduced the idea of local variables on page 27.

Indeed, it is not necessary to use different identifiers for the local variables in *impMakeUpper* and *impMakeUpperTwo*. Nor is it necessary to use different identifiers for the inputs to *MAKEUPPERTWO* and to *MAKEUPPER*. We could equally well have written the implementation of *MAKEUPPERTWO* as below.

```
function MAKEUPPERTWO(c in Char) return in Char
pre   true.
post  If c is a lower-case letter, then the returned value is the letter c in
upper case. If c is not a lower-case letter, then the returned value is c.

{
  var aChar in Char
  aChar  $\leftarrow$  c
  if ((96 <Int ASC(aChar))  $\wedge$  (ASC(aChar) <Int 123)) then
    {aChar  $\leftarrow$  MAKEUPPER(aChar)}
  return aChar
}
```

For code that calls *MAKEUPPER*, the only thing you need to know is its specification. Details of its implementation are not relevant, and that includes the identifiers used both for variables inside that implementation, and for the inputs in the specification. From the point of view of the outside code in *impMakeUpperTwo*, all that happens when *MAKEUPPER* is called is that the appropriate value is calculated and returned. No variables in the outside code have their state changed by the process of calculation. After *MAKEUPPER*(*theChar*) has been calculated for the current state of

theChar, the state of the variable *theChar* will be changed. However, this change is achieved by the assignment

$$theChar \leftarrow MAKEUPPER(theChar)$$

which is part of the outside code. The effect of the outside code is determined by that outside code itself, together with the specification of *MAKEUPPER*, and does not depend on the particular implementation, *impMakeUpper*.

This may sound rather complicated, but it is not really. If you are tracing code that calls some process, look at the specification of the process. That should provide all the information you need in order to be able to trace the code. Usually, there will not be any implementation of the process for you to see, only its specification.

4.3 Testing correctness

It is helpful to include comments on the code in an implementation to show why you think it should be correct. Sometimes, as in Activity 4.2, it is possible to trace code for a general input to be sure that the implementation is correct. This is not usually possible, though. More typically, one will trace an implementation for one or more particular inputs (which satisfy the precondition), and check that the returned value satisfies the postcondition. This is known as **testing** the implementation. Such testing of code does not prove that it will return the correct value in all cases but will often reveal errors. When you have written an implementation, it is sensible to test at least a typical input and any special cases that there might be. (Systematic approaches to testing form an important topic, but not one that will be addressed in this course.)

Activity 4.4

If the implementation *impAddFour* (see Activity 4.1) is correct, what value will it return with *s* equal to: (a) [5,10,3,7]; (b) [5,10,3,7,12]?

Discussion If the code is correct, then the returned value will satisfy the postcondition so long as the input value satisfies the precondition.

- (a) The returned value will be 25, since [5,10,3,7] contains 4 integers and so forms an input of the correct type that satisfies the precondition, and the postcondition states that the returned value is the sum of these four integers: $25 = 5 + 10 + 3 + 7$.
- (b) In this case, the sequence [5,10,3,7,12] contains 5 integers, so does not satisfy the precondition. In such a case, we have no idea what value the code will return, if any. (In saying that code is correct, we are only saying that it works if the precondition is satisfied. For an input that does not satisfy the precondition, anything might happen. The code might produce no returned value, or it might return a value that fails the postcondition. Or it might even happen to return a value that does satisfy the postcondition!)

Activity 4.5

- (a) Give an implementation of the function *ISCAP*, specified below. You may use functions from the following list: *ASC*; $<_{Int}$.

function *ISCAP*(*c* in *Char*) **return** in *Bool*

pre true.

post The returned value is **true** if the character *c* is an upper-case letter and is **false** otherwise.

- (b) A function *COUNTCAPS₄* is specified below, and an implementation *anImp*, which is intended to implement this function, is also given. (The specification of the function *ISCAP* is as given in (a).) Trace *anImp* with each of the inputs: (i) “This”; (ii) “Am I”. Is the implementation *anImp* correct?

Recall that we use □ to show a space character.

function *COUNTCAPS₄*(*s* in *SeqOfChar*) **return** in *Int*

pre The sequence *s* contains exactly four characters.

post The returned value is the number of characters in *s* that are upper-case letters.

```

/* anImp */
{
  var count in Int
  var aSeq in SeqOfChar
  count ← 0
  aSeq ← s
  if (ISCAP(LAST(aSeq))) then
    {count ← count + 1}
  else
    {aSeq ← REMOVELAST(aSeq)}
  if (ISCAP(LAST(aSeq))) then
    {count ← count + 1}
  else
    {aSeq ← REMOVELAST(aSeq)}
  if (ISCAP(LAST(aSeq))) then
    {count ← count + 1}
  else
    {aSeq ← REMOVELAST(aSeq)}
  if (ISCAP(LAST(aSeq))) then
    {count ← count + 1}
  else
    {aSeq ← REMOVELAST(aSeq)}
  return count
}

```

Discussion

- (a) An implementation is given below. Note that upper-case letters have ASCII codes between 65 and 90 inclusive.

```

function ISCAP(c in Char) return in Bool
pre   true.
post  The returned value is true if the character c is an upper-case
letter and is false otherwise.

{
  var b in Bool
  b  $\leftarrow$  true
  if (ASC(c)  $<_{Int}$  65) then {b  $\leftarrow$  false}
  /* Changes b to false if the ASCII code of c is too small. */
  if (90  $<_{Int}$  ASC(c)) then {b  $\leftarrow$  false}
  /* Changes b to false if the ASCII code of c is too large. */
  return b
}

```

(b) Traces are given below.

(i)	Code	Comments	<i>aSeq</i>	<i>count</i>
	<i>count</i> $\leftarrow$ 0		-	0
	<i>aSeq</i> $\leftarrow$ <i>s</i>		"This"	0
	if (<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)))	false		
	<i>aSeq</i> $\leftarrow$ <i>REMOVELAST</i> (<i>aSeq</i>)	use else	"Thi"	0
	if (<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)))	false		
	<i>aSeq</i> $\leftarrow$ <i>REMOVELAST</i> (<i>aSeq</i>)	use else	"Th"	0
	if (<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)))	false		
	<i>aSeq</i> $\leftarrow$ <i>REMOVELAST</i> (<i>aSeq</i>)	use else	"T"	0
	if (<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)))	true		
	<i>count</i> $\leftarrow$ <i>count</i> + 1	use then	"T"	1

The value returned by the code in *anImp* is the final value of *count*, which is 1. The postcondition in the specification states that the value returned should be the number of capital letters in the input string *s*, which is "This". Now "This" contains one capital letter, so the value returned by the code is correct in this case.

(ii)	Code	Comments	<i>aSeq</i>	<i>count</i>
	<i>count</i> $\leftarrow$ 0		-	0
	<i>aSeq</i> $\leftarrow$ <i>s</i>		"Am□I"	0
	if (<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)))	true		
	<i>count</i> $\leftarrow$ <i>count</i> + 1	use then	"Am□I"	1
	if (<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)))	true		
	<i>count</i> $\leftarrow$ <i>count</i> + 1	use then	"Am□I"	2
	if (<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)))	true		
	<i>count</i> $\leftarrow$ <i>count</i> + 1	use then	"Am□I"	3
	if (<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)))	true		
	<i>count</i> $\leftarrow$ <i>count</i> + 1	use then	"Am□I"	4

The value returned by the code in *anImp* is the final value of *count*, which is 4. The postcondition in the specification states that the value returned should be the number of capital letters in *s*, i.e. the number of capital letters in "Am□I". Now "Am□I" contains two capital letters, so the value returned by the code is not correct in this case.

The implementation *anImp* of *COUNTCAPS*₄ given in Activity 4.5 is not correct. If there is any input that satisfies the precondition and for which the code returns the wrong value then the code in an implementation is not correct. To be correct, it must work every time!

The error in *anImp* lies in placing the assignment $aSeq \leftarrow REMOVELAST(aSeq)$ after **else**. This needs to be separate from the conditional. If each occurrence of

```

if (ISCAP(LAST(aSeq))) then
    {count  $\leftarrow$  count + 1}
else
    {aSeq  $\leftarrow$  REMOVELAST(aSeq)}
```

is replaced by the two statements

```

if (ISCAP(LAST(aSeq))) then {count  $\leftarrow$  count + 1}
aSeq  $\leftarrow$  REMOVELAST(aSeq)
```

then we get a correct implementation of *COUNTCAPS*₄. This change ensures that each character in *s* is removed after it has been examined to see whether it is a capital letter.

Objectives for Section 4

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: implementation, testing, correct, abstraction.
- ☐ Trace a given implementation of a specification, and check the value returned by such a trace against the postcondition in the specification to test whether the implementation is correct.
- ☐ Distinguish between testing and proving the correctness of an implementation.
- ☐ In cases where the implementation is similar to code you have already seen:
 - give an implementation of a given specification (using a list of available functions);
 - include comments in the code to explain why you believe it to be correct.

Exercises on Section 4

Exercise 4.1

- (a) Give an implementation of the function *MAKELOWER* specified below. You may use functions from the following list: *ASC*, *CHR*, *+*, *^*, *<Int*.

```

function MAKELOWER(c in Char) return in Char
pre    The character c is an upper-case letter.
post  The returned value is the letter c in lower case. So, for example,
        if c = 'Q' then the returned value is 'q'.
```

- (b) Give a specification and implementation of a function *MAKELOWERTWO*, which changes the case from upper to lower for a character that is an upper-case letter, and leaves other characters unchanged. You can use functions listed in (a), and also *MAKELOWER*.

Exercise 4.2

Look at the discussion of Activity 4.1. Would the code in *impAddFour* that is given there be a correct implementation of the function below?

```

function ADDSEQ(s in SeqOfInt) return in Int
pre    The sequence s contains at least four members.
post  The returned value is the sum of the numbers in s.
```

Exercise 4.3

Give an implementation of the function *COUNTFIVE*, specified below. You may use functions from the following list: *LAST*, *REMOVELAST*, *+*, *=_{Char}*.

function *COUNTFIVE*(*c* in *Char*, *s* in *SeqOfChar*) **return** in *Int*

pre The sequence *s* contains exactly five members.

post The returned value is the number of times that *c* appears in *s*. The returned value is 0 if *c* does not appear in *s*.

5 Repetition in code

In this section, we shall introduce a new construct in the course code. We consider a situation where a statement, or a group of statements, is performed several times in succession. You saw a number of examples of this situation in Section 4. For example, *impAddFour* in the comment on Activity 4.1 involves four repetitions of the assignments:

```
sum ← sum + LAST(aSeq)
aSeq ← REMOVELAST(aSeq).
```

In the course code, such a repetition can be achieved by writing the line

```
for (index ← 1 to 4)
```

before the statements you wish to repeat and enclosing the statements in curly brackets.

Here, *index* is a variable of type *Int*. The statements following this line are repeated four times: first with *index* set as 1; then with *index* set as 2; then with *index* set as 3, and finally with *index* set as 4. Where the statements that are being repeated do not refer to the variable *index*, as is the case in this example, then the effect of **for** (*index* ← 1 **to** 4) is simply to repeat the statements four times. The curly brackets make clear which statements are to be repeated. Thus

```
for (index ← 1 to 4)
{
    sum ← sum + LAST(aSeq)
    aSeq ← REMOVELAST(aSeq)
}
```

has the same effect as the statements

```
sum ← sum + LAST(aSeq)
aSeq ← REMOVELAST(aSeq)
sum ← sum + LAST(aSeq)
aSeq ← REMOVELAST(aSeq)
sum ← sum + LAST(aSeq)
aSeq ← REMOVELAST(aSeq)
sum ← sum + LAST(aSeq)
aSeq ← REMOVELAST(aSeq)
```

We refer to any group of code statements involving repetition as a **loop**. The loop includes the statements from **for** to the closing bracket at the end

Indenting the statements inside the curly brackets is not an essential part of the code. Nor is it essential to put the curly brackets on separate lines. Both are done to make code easier to read.

Imagine a situation where 10000 repetitions are needed. Writing them all out in this way is then not a practical possibility.

of the repeated statements. The statements inside the brackets are called the **body** of the loop.

In the course code, the statement **for** ($index \leftarrow 1$ **to** 4) takes the variable $index$ through the values 1, 2, 3 and 4. There is no need to give an assignment setting $index$ to 1 at the start of the loop, or an assignment $index \leftarrow index + 1$ to increase its value by 1 each time the loop body is executed. Indeed, you should not include any statements assigning a value to $index$. The required changes to $index$ are made by the **for** statement, and no other assignments modifying the value of $index$ should be given.

We refer to $index$ as the **loop variable**. You may use any identifier for the loop variable, but make sure that your choice is different from any other local variables that you are using. To help in avoiding name clashes, we will usually declare all local variables at the start of code, including any used as loop variables in **for** loops. However (for reasons set out below), statements that refer to a loop variable will only be used within the body of the **for** loop.

We are describing how a **for** loop in the course code behaves. Not every programming language interprets a **for** loop in exactly the same way.

Activity 5.1

What is the state of the variable i after the code fragment below has been executed?

```
var i in Int
var index in Int
i ← 7
for (index ← 1 to 5)
{
    i ← i + 2
}
```

Discussion The variable i is first set to 7. The value stored in i is increased by 2 each time the assignment $i \leftarrow i + 2$ is executed. The **for** loop will execute this assignment five times. So the value stored in i after the code fragment has been executed is $7 + 5 \times 2 = 17$. You may find it helpful to trace the variables. When tracing loops we can use a slightly different format.

Comment	$index$	i
$i \leftarrow 7$	-	7
loop	1	9
	2	11
	3	13
	4	15
loop end	5	17

In this format we keep track of key values each time the loop body is executed, but do not show each line of code.

Activity 5.2

Modify the implementation *impCountFive* in the solution to Exercise 4.3 to produce an implementation using a **for** loop.

Discussion The only changes are to replace the repeated statements by a **for** loop and to declare the loop variable.

function *COUNTFIVE*(*c in Char, s in SeqOfChar*) **return in** *Int*

pre The sequence *s* contains exactly five members.

post The returned value is the number of times that *c* appears in *s*. The returned value is 0 if *c* does not appear in *s*.

```

/* impCountFive2 */
{
  var count in Int
  var aSeq in SeqOfChar
  var aChar in Char
  var index in Int
  count ← 0
  aSeq ← s
  for (index ← 1 to 5)
  {
    aChar ← LAST(aSeq)
    if (c =Char aChar) then {count ← count + 1}
    aSeq ← REMOVELAST(aSeq)
  }
  /* count gives the number of copies of the character c
  that have been removed from aSeq. */
}
return count
}

```

Activity 5.3

- (a) Suppose that the function *ISCAP* is as specified in Activity 4.5. Trace the code below, and determine the state of the variable *count* after it has been executed.

```

var count in Int
var aSeq in SeqOfChar
var index in Int
aSeq ← "CaPitaL"
count ← 0
for (index ← 1 to 5)
{
  if (ISCAP(LAST(aSeq))) then {count ← count + 1}
  aSeq ← REMOVELAST(aSeq)
}

```

- (b) Describe informally the effect of the code in (a). To examine all the characters in the string "CaPitaL", how should the code be modified?

Discussion

(a) Following the lines of code step-by-step we obtain the following.

Code	Comment	<i>index</i>	<i>count</i>	<i>aSeq</i>
<i>aSeq</i> ← “CaPitaL”		-	-	“CaPitaL”
<i>count</i> ← 0		-	0	“CaPitaL”
	set <i>index</i> to 1	1	0	“CaPitaL”
	<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)) = true			
	use then	1	1	“CaPitaL”
<i>count</i> ← <i>count</i> + 1		1	1	“CaPita”
<i>aSeq</i> ← <i>REMOVELAST</i> (<i>aSeq</i>)		2	1	“CaPita”
	set <i>index</i> to 2			
	<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)) = false			
	do not use then	2	1	“CaPita”
<i>aSeq</i> ← <i>REMOVELAST</i> (<i>aSeq</i>)		2	1	“CaPit”
	set <i>index</i> to 3	3	1	“CaPit”
	<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)) = false			
	do not use then	3	1	“CaPit”
<i>aSeq</i> ← <i>REMOVELAST</i> (<i>aSeq</i>)		3	1	“CaPi”
	set <i>index</i> to 4	4	1	“CaPi”
	<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)) = false			
	do not use then	4	1	“CaPi”
<i>aSeq</i> ← <i>REMOVELAST</i> (<i>aSeq</i>)		4	1	“CaP”
	set <i>index</i> to 5	5	1	“CaP”
	<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>)) = true			
	use then	5	2	“CaP”
<i>count</i> ← <i>count</i> + 1		5	2	“Ca”
<i>aSeq</i> ← <i>REMOVELAST</i> (<i>aSeq</i>)		5	2	“Ca”

However, we do not need to write out the trace table in so much detail. The trace table below shows the value stored in the variables after each execution of the loop body. It also shows the value of the condition that determines whether *count* is incremented in the next pass through the loop.

Comment	<i>index</i>	<i>count</i>	<i>aSeq</i>	<i>ISCAP</i> (<i>LAST</i> (<i>aSeq</i>))
<i>aSeq</i> ← “CaPitaL”	-	0	“CaPitaL”	true
loop	1	1	“CaPita”	false
	2	1	“CaPit”	false
	3	1	“CaPi”	false
	4	1	“CaP”	true
loop end	5	2	“Ca”	false

This will be the format expected for trace tables of loops in your answers to questions.

(b) The effect of the code is to examine the last five characters in the string “CaPitaL”, and discover how many of these are upper-case letters. The resulting value is stored in the variable *count*. (As each character is examined, it is removed from *aSeq*. After the code is executed, *aSeq* contains any remaining characters from “CaPitaL” that have not been examined.)

To ensure that all the characters in “CaPitaL” are examined, the statement **for** (*index* ← 1 **to** 5) should be modified by changing the 5 to 7 (the number of characters in “CaPitaL”).

The statements in the body of a **for** loop may refer to the loop variable. If the statements being repeated do include any reference to the loop variable, then the change in its value needs to be followed, as in Activity 5.4 below.

In the course code, statements outside a **for** loop should not refer to the loop variable. This is a safety precaution. If you use the loop variable for

other purposes outside the loop, then the **for** loop may overwrite data you wished to keep. After the **for** loop, you cannot be sure exactly what value may be stored by the loop variable (since different programming languages may implement a **for** loop in slightly different ways). So in the course code, we regard the value stored in the loop variable directly after a **for** loop as indeterminate (just like the value stored by a local variable immediately after a **var** statement, before it is initialised).

Activity 5.4

- (a) Trace the code below, and determine the state of the variable *sum* after the code has been executed.

```
var sum in Int
var index in Int
sum ← 0
for (index ← 1 to 5)
{
    sum ← sum + index
}
```

- (b) Describe informally the effect of the code in (a).

Discussion

- (a) A trace is given below.

Comment	<i>index</i>	<i>sum</i>
<i>sum</i> ← 0	-	0
loop	1	1
	2	3
	3	6
	4	10
loop end	5	15

- (b) The effect of the code is to add together the numbers 1, 2, 3, 4 and 5, and to store the resulting value in *sum*.

The values taken by the loop variable in a **for** loop do not have to start at 1. For example, in Activity 5.4, if **for** (*index* ← 1 to 5) were replaced by **for** (*index* ← 4 to 8), then the effect of the code would be to add up the numbers 4, 5, 6, 7 and 8.

You have seen a number of pieces of code which perform some task for a sequence that contains a particular number of members. For example, *impCountFive2* in Activity 5.2 counts the number of times a given character appears in a string, where the string contains exactly five characters. We can reuse more or less the same code for different sized strings, by replacing the 5 in **for** (*index* ← 1 to 5) with the appropriate value. But it is not convenient to have a number of different functions, say *COUNTFOUR*, *COUNTFIVE*, *COUNTSIX*, and so on. Any code wanting to use these would need to know the number of characters in a string before it knew which function to call. We gave such functions as examples earlier because you had not then met the code constructs needed for a more general approach. Using the code construct **for**, the more general approach is possible. To write such code, we also need a function to be available that returns the number of members in a sequence. A suitable specification is given below.

function *SIZE*(*s* in SeqOfX) **return** in Int
pre true.

SIZE(*s*) is the same as the length of the sequence *s*.

post The returned value is the number of items in the sequence *s*. If *s* contains no members then the returned value is 0.

If *SIZE* is available, then we can extend *impCountFive2* to a more general context. We use *SIZE*(*s*) repetitions, and make a suitable modification to the specification of the function that is being implemented.

function *CHARINSEQ*(*c* in Char, *s* in SeqOfChar) **return** in Int
pre true.

post The returned value is the number of times that *c* appears in *s*. If *c* does not appear in *s* then the returned value is 0. If *s* is the empty sequence then the returned value is 0.

```
/* impCharInSeq */
{
  var count in Int
  var aSeq in SeqOfChar
  var aChar in Char
  var index in Int
  count ← 0
  aSeq ← s
  for (index ← 1 to SIZE(s))
  {
    aChar ← LAST(aSeq)
    if (c =Char aChar) then {count ← count + 1}
    aSeq ← REMOVELAST(aSeq)
  }
  /* count gives the number of copies of the character c
  that have been removed from aSeq. */
}
return count
}
```

If the sequence *s* input to this code has no members, then *SIZE*(*s*) = 0. In this case, the **for** statement in *impCharInSeq* becomes

for (*index* ← 1 to 0)

You may reasonably ask: “What happens in this case?” The answer is: “The loop is omitted entirely.” In general, the statements in the body of a loop following

for (*index* ← *m* to *n*)

are repeated with *index* equal to, in turn, *m*, then *m* + 1, then *m* + 2, and so on, until *index* is equal to *n*. This description of what happens makes sense as long as *m* ≤ *n*. (If *m* = *n*, then the loop body is executed just once, with *index* = *m* = *n*.) If *m* > *n*, then the statements in the body of the loop are never executed.

Another point to notice in *impCharInSeq* is that *SIZE*(*s*) in the **for** condition is a reference to the input sequence. We want to repeat the loop body once for each member of the input sequence, *s*. Although the local variable *aSeq* is initially set to *s*, the value of *aSeq* varies as the loop is executed. In a **for** condition, we will avoid use of any quantity that varies as the loop is executed. This is to avoid uncertainty about how often the loop is to be executed.

Activity 5.5

The implementation *impAddFour* in the comment on Activity 4.1 evaluates a function *ADDFOUR* that sums the numbers in a sequence of numbers containing exactly four members. Give a specification of a function *SUMSEQ* that sums the members of any sequence of numbers. Then give an implementation of *SUMSEQ*. You may use functions from the following list: *LAST*, *REMOVELAST*, *+* and *SIZE*.

Discussion A specification of *SUMSEQ* is given below. The semantics are easily interpreted for any sequence that is not empty, but we need to explain what is to happen in the special case where the input sequence has no members. We could have excluded this case using a precondition, but we have chosen to interpret the “sum of no numbers” as 0.

To obtain an implementation, we can modify *impAddFour* to use **for**, and to repeat the statements in the body of the loop once for each member of the input sequence. In the special case where *s* contains no members, the statements in the body of the **for** loop are never executed, so *sum* still stores its initial value of 0 after the loop.

function *SUMSEQ*(*s* in *SeqOfInt*) **return in** *Int*
pre **true**.
post The returned value is the sum of the numbers in *s*. If *s* is empty then the returned value is 0.

```
{
  var sum in Int
  var aSeq in SeqOfInt
  var index in Int
  aSeq ← s
  sum ← 0
  for (index ← 1 to SIZE(s))
  {
    sum ← sum + LAST(aSeq)
    aSeq ← REMOVELAST(aSeq)
  }
  /* sum gives the total of the numbers that have been
  removed from aSeq. */
  return sum
}
```

Objectives for Section 5

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: loop; loop body; loop variable.
- ☐ Interpret and use the notation: **for** (*index* ← *m* to *n*).
- ☐ Trace a code fragment involving **for** loops.

Exercises on Section 5

Exercise 5.1

Trace the code below, and determine the state of the variable *count* after it has been executed.

```

var count in Int
var index in Int
count  $\leftarrow$  0
for (index  $\leftarrow$  2 to 5)
{
    count  $\leftarrow$  count + index  $\times$  index
}
count  $\leftarrow$  count + 1

```

Exercise 5.2

What will be the value stored by the variable *count* after the loop below has been executed? Your answer should cover all cases for the values of *m* and *n*.

```

var count in Int
var index in Int
count  $\leftarrow$  0
for (index  $\leftarrow$  m to n)
{
    count  $\leftarrow$  count + 1
}

```

Exercise 5.3

The function *COUNTCAPS* inputs a string and returns the number of capital letters that it contains. Give a specification and implementation of *COUNTCAPS*. You may use functions from the following list: *ISCAP* (see Activity 4.5); *LAST*; *REMOVELAST*; *SIZE*; +.

Summary of Unit 2

1. The course code provides a way of expressing statements in a form suitable for interpretation by a computer. In the course code, all variables must be given a type, through a **var** statement. For now, variables are not created with default values, so must be initialised (this issue is discussed further later in the course). A trace follows the values of variables as code is executed.
2. The course code includes conditional statements, of the form **if** ($\dots$) **then** { $\dots$ } or **if** ($\dots$) **then** { $\dots$ } **else** { $\dots$ }. The statements to be executed after **then** or **else** are enclosed in curly brackets, as {statements}.
3. Code may call on processes. To trace the effect of code that calls on a computing process, you can refer to the specification of that process. We considered only functions; that is, processes that return a value. You saw how a function can be specified in the course code. A process specified as **function** (**infix**) is called in the course code using infix notation.

4. Code that gives details of how the value returned by a function may be calculated is called an implementation of the function specification. You saw how the course code expresses an implementation. When you call a function in the course code, the implementation is not visible. You only need to see the specification of the function. You then know the effect of the code being called — which is all you need to know — but not the details of the code being called.

Code implementing a specification needs to be correct. To say that code is correct means that, for any input of the appropriate type that satisfies the precondition in the specification, the value returned by the code must satisfy the postcondition in the specification. So long as an implementation is correct, then to see the effect of the code, you only need to refer to the specification.

For simple code, it may be possible to trace it for a general input, to verify that the code is correct in all cases. More typically, one will test the code by tracing it for sample inputs. Testing cannot prove that an implementation is correct, but will often reveal errors. Testing shows that the code in an implementation is not correct if you find any one input value that satisfies the precondition, and for which the value returned by the code in the implementation does not satisfy the postcondition in the specification.

5. In code, we use a loop to show that a group of statements is to be executed several times. A loop of the form

```

for (index  $\leftarrow$  m to n)
{
    loop body
}

```

repeats the statements in loop body. These statements are repeated with *index* set equal to *m*, then equal to *m* + 1, then equal to *m* + 2, up to *index* equal to *n*. (If *n* is less than *m*, then the statements in loop body are not executed at all.) If the statements within loop body do not refer to *index*, then **for** (*index* $\leftarrow$ 1 **to** *n*) {loop body} just repeats the statements in loop body *n* times.

Solutions to the exercises

Solution 1.1

A trace is given below:

	<i>varOne</i>	<i>varTwo</i>	<i>count</i>
<i>varOne</i> $\leftarrow$ 14	14	-	-
<i>varTwo</i> $\leftarrow$ 5	14	5	-
<i>count</i> $\leftarrow$ 0	14	5	0
<i>varOne</i> $\leftarrow$ <i>varOne</i> - <i>varTwo</i>	9	5	0
<i>count</i> $\leftarrow$ <i>count</i> + 1	9	5	1
<i>varOne</i> $\leftarrow$ <i>varOne</i> - <i>varTwo</i>	4	5	1
<i>count</i> $\leftarrow$ <i>count</i> + 1	4	5	2

Solution 2.1

(a) A trace with n in the state 16 at the start is given below.

Code	Comment	n	$aBool$
$aBool \leftarrow \text{true}$		16	true
if ($MOD(n, 2) =_{Int} 0$)	($MOD(16, 2) =_{Int} 0$) is true		
$n \leftarrow DIV(n, 2)$	so use then	8	true
if ($MOD(n, 3) =_{Int} 0$)	($MOD(8, 3) =_{Int} 0$) is false		
$aBool \leftarrow \text{false}$	so use else	8	false
if ($MOD(n, 2) =_{Int} 0$)	($MOD(8, 2) =_{Int} 0$) is true		
$n \leftarrow DIV(n, 2)$	so use then	4	false

(b) A trace with n in the state 24 at the start is given below.

Code	Comment	n	$aBool$
$aBool \leftarrow \text{true}$		24	true
if ($MOD(n, 2) =_{Int} 0$)	($MOD(24, 2) =_{Int} 0$) is true		
$n \leftarrow DIV(n, 2)$	so use then	12	true
if ($MOD(n, 3) =_{Int} 0$)	($MOD(12, 3) =_{Int} 0$) is true		
$n \leftarrow DIV(n, 3)$	so use then	4	true
if ($MOD(n, 2) =_{Int} 0$)	($MOD(4, 2) =_{Int} 0$) is true		
$n \leftarrow DIV(n, 2)$	so use then	2	true

Note that $aBool$ gets set to **false** if n is not divisible by $2 \times 3 \times 2 = 12$.

Solution 2.2

(a) (i) A trace with c in the state 'J' at the start is given below.

Code	Comment	n	$aBool$
$aBool \leftarrow \text{false}$		-	false
$n \leftarrow ASC(c)$		74	false
if ($64 <_{Int} n$)	($64 <_{Int} n$) is true		
	so use then		
if ($n <_{Int} 91$)	($n <_{Int} 91$) is true		
$aBool \leftarrow \text{true}$	so use then	74	true
if ($96 <_{Int} n$)	($96 <_{Int} n$) is false		
	no else , so end of code	74	true

(ii) A trace with c in the state 'j' at the start is given below.

Code	Comment	n	$aBool$
$aBool \leftarrow \text{false}$		-	false
$n \leftarrow ASC(c)$		106	false
if ($64 <_{Int} n$)	($64 <_{Int} n$) is true		
	so use then		
if ($n <_{Int} 91$)	($n <_{Int} 91$) is false		
	no else , so no change	106	false
if ($96 <_{Int} n$)	($96 <_{Int} n$) is true		
	so use then		
if ($n <_{Int} 123$)	($n <_{Int} 123$) is true		
$aBool \leftarrow \text{true}$	so use then	106	true

(iii) A trace with c in the state '7' at the start is given below.

Code	Comments	n	$aBool$
$aBool \leftarrow \text{false}$		-	false
$n \leftarrow ASC(c)$		55	false
if ($64 <_{Int} n$)	($64 <_{Int} n$) is false		
	no else , so no change	55	false
if ($96 <_{Int} n$)	($96 <_{Int} n$) is false		
	no else , so no change	55	false

(b) The first conditional will change $aBool$ to **true** if both the conditions ($64 <_{Int} n$) and ($n <_{Int} 91$) are **true**. Since n is the ASCII code of c , this will be the case if c

is an upper-case letter. The second conditional will change *aBool* to **true** if both the conditions $(96 <_{Int} n)$ and $(n <_{Int} 123)$ are **true**. Since *n* is the ASCII code of *c*, this will be the case if *c* is a lower-case letter. The code will set *aBool* to **true** if *c* is a letter character (in either lower or upper case). If *c* is a character that is not a letter, then the code will set *aBool* to **false**.

Solution 2.3

A hint as to how to approach this can be obtained if you look back to Activity 2.2(d) to see the effect of the code in Fragment 3. That changed a lower-case character into the corresponding upper-case character. The necessary modifications to reverse this process are built into the code below.

```
var num in Int
num ← ASC(aChar)
if ((64 <Int num) ∧ (num <Int 91)) then
{
    num ← num + 32
    aChar ← CHR(num)
}
```

The condition after **if** is now **true** if *aChar* is an upper-case character when the code is encountered. To go from upper to lower case, we need to add 32 to the ASCII code.

An equally good alternative answer can be obtained by modifying Fragment 4 (which uses nested conditionals), as below.

```
var num in Int
num ← ASC(aChar)
if (64 <Int num) then
{
    if (num <Int 91) then
    {
        num ← num + 32
        aChar ← CHR(num)
    }
}
```

Solution 3.1

- (a) **function** *ADDLAST*(*x* in *X*, *s* in *SeqOfX*) **return** in *SeqOfX*
pre true.
post The returned value is the sequence formed by adding *x* to the end of the sequence *s*. So, for example, *ADDLAST*(5, [−3, 2, 4]) = [−3, 2, 4, 5].
- (b) **function** *INTHREE*(*c* in *Char*, *s* in *SeqOfChar*) **return** in *Bool*
pre The sequence *s* contains exactly three members.
post The returned value is **true** if *c* appears in *s* (one or more times), and is **false** otherwise.
- (c) **function** (**infix**) (*a* =_{Char} *b* **on** *Char*) **return** in *Bool*
pre true.
post The returned value is **true** if the characters *a* and *b* are the same and is **false** otherwise.

Solution 3.2

After each application of *REMOVELAST*, *aSeq* contains one less character: “abC”, then “ab”, then “a”. So after three applications of the assignment *aSeq* ← *REMOVELAST*(*aSeq*), *aSeq* stores the empty sequence []. (Since this is in fact the empty string, we could have used the notation “”.) This does not satisfy the precondition of *REMOVELAST*, so the final call of *REMOVELAST*(*aSeq*) is not valid.

Solution 4.1

- (a) You can use an implementation similar to that for *MAKEUPPER*.

function *MAKELOWER*(*c in Char*) **return in** *Char*
pre The character *c* is an upper-case letter.
post The returned value is the letter *c* in lower case. So, for example, if *c* = 'Q' then the returned value is 'q'.

```
{
  var num in Int
  var aChar in Char
  aChar ← c
  num ← ASC(aChar)
  num ← num + 32
  aChar ← CHR(num)
  return aChar
}
```

You might have used the single assignment $num \leftarrow ASC(c)$ instead of the two assignments $aChar \leftarrow c$ and $num \leftarrow ASC(aChar)$. This is correct as well.

- (b) You could use nested conditionals, in a way similar to Fragment 4 in Section 1. Alternatively, you can use an approach similar to that used for *MAKEUPPERTWO* in the text.

function *MAKELOWERTWO*(*c in Char*) **return in** *Char*
pre true.
post If *c* is an upper-case letter, then the returned value is the letter *c* in lower case. If *c* is not an upper-case letter, then the returned value is *c*.

```
{
  var aChar in Char
  aChar ← c
  if ((64 <Int ASC(aChar)) ∧ (ASC(aChar) <Int 91)) then
  {
    aChar ← MAKELOWER(aChar)
  }
  return aChar
}
```

Solution 4.2

The code in *impAddFour* would not form a correct implementation of the function *ADDSEQ*. Suppose that $s = [1, 2, 3, 4, 5]$, for example. This sequence satisfies the precondition of *ADDSEQ*. The code in *impAddFour* will return the value obtained by adding the last four numbers in this sequence; that is, $2 + 3 + 4 + 5 = 14$. The sum of the numbers in s is $1 + 2 + 3 + 4 + 5 = 15$, so the returned value fails to satisfy the postcondition.

Note that we have shown that the implementation is incorrect by giving a specific example where it fails. This is the simplest and most convincing way to explain why an implementation is not correct.

Solution 4.3

We need to compare each of the five characters in the string s with c . An implementation is given below. After each occurrence of the assignment

$aSeq \leftarrow REMOVELAST(aSeq)$

one could add the comment:

/* count gives the number of copies of the character c that have been removed from $aSeq$. */

function *COUNTFIVE* (*c* in *Char*, *s* in *SeqOfChar*) **return** in *Int*
pre The sequence *s* contains exactly five members.
post The returned value is the number of times that *c* appears in *s*. The returned value is 0 if *c* does not appear in *s*.

```
/* impCountFive */
{
  var count in Int
  var aSeq in SeqOfChar
  var aChar in Char
  count ← 0
  aSeq ← s
  aChar ← LAST(aSeq)
  if (c =Char aChar) then {count ← count + 1}
  aSeq ← REMOVELAST(aSeq)
  aChar ← LAST(aSeq)
  if (c =Char aChar) then {count ← count + 1}
  aSeq ← REMOVELAST(aSeq)
  aChar ← LAST(aSeq)
  if (c =Char aChar) then {count ← count + 1}
  aSeq ← REMOVELAST(aSeq)
  aChar ← LAST(aSeq)
  if (c =Char aChar) then {count ← count + 1}
  aSeq ← REMOVELAST(aSeq)
  aChar ← LAST(aSeq)
  if (c =Char aChar) then {count ← count + 1}
  aSeq ← REMOVELAST(aSeq)
  return count
}
```

Use of the variable *aChar* here is not essential. You could equally well replace each occurrence of the two statements

```
aChar ← LAST(aSeq)
if (c =Char aChar) then {count ← count + 1}
```

by the single statement

```
if (c =Char LAST(aSeq)) then {count ← count + 1}
```

Solution 5.1

A trace of the code is given below. The final state of *count* is 55.

Comment	<i>index</i>	$index \times index$	<i>count</i>
<i>count</i> ← 0	-	-	0
loop	2	4	4
	3	9	13
	4	16	29
	5	25	54
loop end			
<i>count</i> ← <i>count</i> + 1	-	-	55

Solution 5.2

Since *count* starts as 0, and increases by 1 each time the loop body is executed, it will end up equal to the number of times that the loop body has been executed. If $m > n$, the loop is not executed at all. If $m = n$, the loop is executed once. If $m < n$, the loop is executed $n - m + 1$ times. (To check this, try an example, such as $m = 4$ and $n = 6$. The loop is executed with *index* equal to 4, 5 and 6; that is, three times.) So after the loop, the variable *count* stores the value:

```
0          if  $m > n$ 
1          if  $m = n$ 
 $n - m + 1$  if  $m < n$ .
```

Solution 5.3

If you would like a hint, look at Activity 5.3. We can modify the code there to look at all the characters in a general string. In the specification, it is sensible to make clear what is the returned value when the input string is empty. Since an empty string contains no capitals, it is natural to return 0 in that case.

function *COUNTCAPS*(*s* in *SeqOfChar*) **return in** *Int*

pre true.

post The returned value is the number of characters in *s* that are upper-case letters. If *s* is empty, the returned value is 0.

```
{
  var count in Int
  var aSeq in SeqOfChar
  var index in Int
  aSeq ← s
  count ← 0
  for (index ← 1 to SIZE(s))
  {
    if (ISCAP(LAST(aSeq))) then {count ← count+1}
    aSeq ← REMOVE LAST(aSeq)
  }
  /* count gives the number of upper-case letters that have been
  removed from aSeq. */
}
return count
}
```

Index

←, 5

abstraction, 4, 29

assignment

statement, 5

symbol, 5

code

invalid, 11

tracing, 8

comment, 9

conditional, 12

course code, 4

data type, 7

default, 7

function, 17

header, 25

identifier, 5

imperative language, 4

implementation, 4

correct, 25

infix, 20

initialisation, 7

literal, 6

loop, 35

body, 36

variable, 36

postcondition, 17

precondition, 17

programming language, 4

reserved word, 11

specification, 4, 17

statement

else, 12

for, 35

if, 12

var, 7

assignment, 5

testing, 31

trace, 8

type, 7

type consistent, 11

variable, 5

local, 27